PLAYCENT

SMART CONTRACT AUDIT

ZOKYO.

Mar 5, 2021 | v. 1.0

TABLE OF CONTENTS

Auditing Strategy and Techniques Applied 2
Structure and Organization of Document 3
Complete Analysis 4

AUDITING STRATEGY AND TECHNIQUES APPLIED

The Smart contract's source code was taken from the following github repository – https://github.com/jaykishan007/playcent-ico-contract.
Commit id – b0ded09666cc5a4bc13b4250eb1ccba100afb6f4.

Throughout the review process, care was taken to ensure that the token contract:

- Implements and adheres to existing Token standards appropriately and effectively;
- Documentation and code comments match logic and behavior;
- Distributes tokens in a manner that matches calculations;
- Follows best practices in efficient use of gas, without unnecessary waste;
- Uses methods safe from reentrance attacks;
- Is not affected by the latest vulnerabilities;
- Meets best practices in code readability, etc.

Zokyo's Security Team has followed best practices and industry-standard techniques to verify the implementation of Playcent smart contracts. To do so, the code is reviewed line-by-line by our smart contract developers, documenting any issues as they are discovered. In summary, our strategies consist largely of manual collaboration between multiple team members at each stage of the review:

1	Due diligence in assessing the overall code quality of the codebase.	3	Testing contract logic against common and uncommon attack vectors.
2	Cross-comparison with other, similar smart contracts by industry leaders.	4	Thorough, manual review of the codebase, line-by-line.

STRUCTURE AND ORGANIZATION OF DOCUMENT

For ease of navigation, sections are arranged from most critical to least critical. Issues are tagged “Resolved” or “Unresolved” depending on whether they have been fixed or addressed. Furthermore, the severity of each issue is written as assessed by the risk of exploitation or other unexpected or otherwise unsafe behavior:

Critical

The issue affects the ability of the contract to compile or operate in a significant way.

High

The issue affects the ability of the contract to compile or operate in a significant way.

Medium

The issue affects the ability of the contract to operate in a way that doesn’t significantly hinder its behavior.

Low

The issue has minimal impact on the contract’s ability to operate.

Informational

The issue has no impact on the contract’s ability to operate.

COMPLETE ANALYSIS

HIGH | RESOLVED

Contract is not allocating tokens for vestings.

Recommendation:
During vesting allocation allocate tokens for that vesting by transferring tokens to contract address.

HIGH | RESOLVED

Contract owner can withdraw vesting tokens any time that can result in stopping vesting functionality to function.

Recommendation:
Contract owner should be able to withdraw tokens that are not allocated for vesting:

- Increase total vesting amount on every adding of vesting allocation
- Decrease total vesting amount on every vesting claim.

HIGH | RESOLVED

Vested tokens distributions starts in one more after lock period is over.

Recommendation:
Clearly state in public documentation that vested tokens will be available after lock period plus one month or change calculation of month valid for vesting calculation.

MEDIUM | RESOLVED

Method `getTgeTIME` points to `Sat Mar 06 2021 08:12:59 GMT+0000`, but comment points to Sat Mar 06 2021 11:30:00 GMT+0000.

Recommendation:
Fix comment to point to proper date or remove it.

MEDIUM | RESOLVED

Method _claimVestTokens requires distribute tokens amount to be greater than tokens amount allocated on contract address as defined in the snippet:

```
uint256 contractTokenBalance = balanceOf(address(this));
require(
 contractTokenBalance > tokensToTransfer,
 "Not Enough Token Balance in Contract"
);
```

Recommendation:
Let caller to withdraw amount less or equal to balance available to contract itself.

MEDIUM | RESOLVED

Only contract owner is able to trigger tokens distribution for any address.

Recommendation:
Let any user to call methods claimTGETokensOnlyOwner, claimVestTokensOnlyOwner to claim any address & vesting as participants can use cold wallets and send transactions from another addresses.

MEDIUM | RESOLVED

Contract owner can pass any number in range of 0 till 256 for vesting type during call of method `addVestingDetails`.

Recommendation:
Allow to pass only valid vesting type by implementing any of options:

- check existing VestType structure (e.g. check if totalTokenAllocation greater than 0);
- add new attribute to structure to prove that is valid after getting from mapping;
- verify that passed vestType is whitelisted range (_vestnigType >= 0 && _vestnigType <= 9).

LOW | RESOLVED

Smart contract is not covered by NatSpec annotations.

Recommendation:
Cover by NatSpec all Contract methods.

LOW | RESOLVED

Modifier checkVestingStatus has incorrect error message `User NOT added to any Vesting Category` misleads method caller as modifier checks only specific vesting allocation, but not all.

Recommendation:
Use proper error message that represent that passed userAddress has no such vesting allocation.

We are grateful to have been given the opportunity to work with the Playcent team.

The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

Zokyo's Security Team recommends that the Playcent team put in place a bug bounty program to encourage further analysis of the smart contract by third parties.

ZOKYO.